system is very simple because you only need to download the Waf executable file. You can use the following Linux command *wget* to download the Waf executable file if you have Internet connectivity:

```
wget https://waf.io/waf-1.9.6
```

The command *wget* will download the Waf executable file to your current working directory. You can run this executable file with the command *'./waf-1.9.6'*. But remember to rename the file as *waf* and copy this file to any one of the directories pointed by the PATH environment variable of Linux to make matters easy. This step is important because the Linux environment will search for executable files only in the directories pointed by the environment variable PATH and, in most cases, the current working directory will not be included. Execute the command *'echo $PATH'* to see the various directories searched by the Linux Shell for executables. Also remember to change the access permissions by using the command *chmod*. In this case, the command *'chmod 755 waf'* or *'chmod 755 waf-1.9.6'* (depending on whether you have renamed the Waf executable or not) will be sufficient. This command gives read, write and execute permissions to the owner, and gives read and execute permissions for the group and others.

If you do not have Internet connectivity, just download the Waf executable file from the URL *https://waf.io* and copy it to one of the directories pointed by the environment variable PATH, as mentioned earlier. But do remember that this is not the way we are supposed to use Waf. The developers of Waf want us to copy its executable into the main directory of each and every project we are developing along with a Waf script to build the project. This is the single greatest advantage of Waf over most other build automation tools. If you are using the tool Make for building your project, then the target system should have Make installed in it. But with Waf, you do not have this constraint because all you have to do is to bundle an executable of Waf along with the source files of your project. But while writing this article, I primarily had students in mind—those who are trying to customise their systems with a build automation tool and hence chose the method we are discussing.

Waf can be built from the source files but I prefer the easier method, whereby the executable of Waf is downloaded. As an aside, Waf can also be used in MS Windows and the working of the executable is similar in Windows and Linux.

A sample program to test Waf

In order to demonstrate the working of Waf, a simple C program is used in this article. But do remember that Waf can be used to build projects developed in languages like C++, C#, Java, Fortran, Python, D, etc. The simple C program is divided into three files — *main.c*, *data.c* and *show.c*. The file *main.c*, shown below, calls two functions *data( )* and *show( )*. But the function definitions are given in two separate files.

```
void data( );
void show( );
int main( )
{
        data( );
        show( );
        return 0;
}
```

The file *data.c* given below contains the definition of the function *data( )*.

```
#include <stdio.h>
void data( )
{
        printf("\nLet us learn about Waf\n");
}
```

The file *show.c* given below contains the definition of the function *show( )*.

```
#include <stdio.h>
void show( )
{
        printf("\nWaf is a build automation tool\n");
}
```

These three files can be compiled using the C compiler *gcc* from the GNU Compiler Collection (GCC) to generate an executable called *hello* by executing the following single command:

```
gcc main.c data.c show.c -o hello
```

But the problem with this sort of compilation is that all the three files are recompiled irrespective of whether they are modified or not. So we need the following sequence of commands, which will compile each file individually rather than compiling all the files at a single go, in order to avoid unnecessary recompilation of unmodified files.

```
gcc -c main.c
gcc -c data.c
gcc -c show.c
gcc main.o data.o fun.o -o hello
```

The option *–c* of *gcc* tells the compiler to produce object code rather than executable code. But the option *–c*